

DESARROLLO DE SOFTWARE

Laboratorio 1 - Semana 2

Abril 10, 2026

Mera Amado, Ivett Marinella

Prof. Manuel Quispe

Nota: Los archivos de entrada y salida se encuentran en las carpetas input/ y output/ respectivamente, dentro de la raíz del proyecto. La estructura del proyecto es la siguiente:

```
Tarea-1/  
├── src/  
│   ├── P1.py  
│   └── P2.py  
├── input/  
│   ├── p1.txt  
│   └── p2.txt  
└── output/  
    ├── p1.txt  
    └── p2.txt
```

PROBLEMA 1

El problema consiste en simular el sistema de enrutamiento de una Single Page Application. Dadas un conjunto de rutas con su contenido asociado y una secuencia de URLs de transición, se debe determinar qué contenido mostrar para cada URL, considerando que las rutas pueden contener parámetros dinámicos como :id. Para resolverlo, se convierten las rutas con parámetros en expresiones regulares (([^\s/]+)), luego cada URL se compara contra esos patrones en orden. Si hay coincidencia se extrae el parámetro capturado y se imprime el contenido asociado; si ninguna ruta coincide, se imprime 404 Not Found.

Solución

```
import re  
import os  
  
BASE_DIR = os.path.dirname(os.path.abspath(__file__))  
in_path = os.path.join(BASE_DIR, '..', 'input', 'p1.txt')  
out_path = os.path.join(BASE_DIR, '..', 'output', 'p1.txt')  
  
def router():  
    with open(in_path, 'r') as f_in:  
        n = int(f_in.readline().strip())  
        routes = []  
        for _ in range(n):  
            parts = f_in.readline().strip().split(' ', 1)  
            routes.append((parts[0], parts[1]))  
        m = int(f_in.readline().strip())
```

```

results = []
for _ in range(m):
    url = f_in.readline().strip()
    matched = False
    for path, content in routes:
        pattern = re.sub(r'(:([^\s/]+)', r'([^\s/]+)', path)
        pattern = '^' + pattern + '$'
        match = re.match(pattern, url)
        if match:
            param = match.group(1) if match.lastindex else None
            results.append(content + (' ' + param if param else ''))
            matched = True
            break
    if not matched:
        results.append('404 Not Found')

with open(out_path, 'w') as f_out:
    f_out.write('\n'.join(results) + '\n')

if __name__ == '__main__':
    router()

```

PROBLEMA 2

El problema plantea que un banco tiene socios con terminales de pago, y se desea identificar al cliente más fiel de cada socio, o sea, el que realizó la mayor cantidad de compras a través de sus terminales. En caso de empate se elige al cliente con menor ID, y si un socio no tiene compras se imprime -1.

Para resolverlo, se construyen dos estructuras: un diccionario que mapea cada terminal a su socio, y otro que acumula el conteo de compras por cliente para cada socio. Al procesar cada transacción se identifica el socio dueño del terminal y se incrementa el contador del cliente. Finalmente, por cada socio se selecciona el cliente con mayor conteo aplicando un criterio de desempate por menor ID.

Solución

```

import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
in_path = os.path.join(BASE_DIR, '..', 'input', 'p2.txt')
out_path = os.path.join(BASE_DIR, '..', 'output', 'p2.txt')

def banco():
    with open(in_path, 'r') as f_in:
        n, m, s = map(int, f_in.readline().split())

        terminal_to_partner = {}
        for _ in range(m):
            p, t = map(int, f_in.readline().split())
            terminal_to_partner[t] = p

```

```
purchases = {p: {} for p in range(1, n + 1)}

for _ in range(s):
    c, t = map(int, f_in.readline().split())
    if t in terminal_to_partner:
        p = terminal_to_partner[t]
        purchases[p][c] = purchases[p].get(c, 0) + 1

results = []
for p in range(1, n + 1):
    if not purchases[p]:
        results.append(f'{p} -1')
    else:
        best = min(purchases[p], key=lambda c: (-purchases[p][c], c))
        results.append(f'{p} {best}')

with open(out_path, 'w') as f_out:
    f_out.write('\n'.join(results) + '\n')

if __name__ == '__main__':
    banco()
```